GuardAIN: Centralized AI-assisted financial-fraud defence platform

Final-year project presentation and implementation guide

Prepared for academic demonstration | GuardAIN centralized fraud-triage platform

1. Project summary

GuardAIN is a privacy-aware fraud-detection platform designed to help people identify suspicious messages, risky URLs, UPI/VPA payment lures, and credential-phishing webpages. It combines a browser extension, a mobile-style web triage interface (PWA), an AI-assisted FastAPI backend, a central analyst portal, and a shared domain intelligence database.

The central idea is simple: a user-facing component detects corroborated fraud signals, sends a privacy-minimised report to a secure central server, and the analyst portal turns those reports into actionable domain intelligence. A confirmed malicious domain can then protect other connected devices.

GuardAIN is an academic prototype for **analyst-assisted fraud prevention**. It must not be presented as government-approved, a bank system, or a proven replacement for police, bank, or cybercrime investigations.

2. Problem statement

Digital financial fraud increasingly uses:

- fake bank, RBI, CBI, courier, KYC, and government impersonation;
- "digital arrest" threats and urgency;
- OTP, UPI PIN, screen-sharing, remote-app, and credential theft;
- malicious look-alike URLs and cloned login/payment pages;
- Hinglish and regional-language messages that are harder for simple English-only keyword filters to understand.

Traditional filters often have two weaknesses: they either block harmless awareness content because it contains words such as "OTP", or they do not combine language, page behaviour, domain reputation, and analyst feedback.

3. Proposed solution

GuardAIN uses layered evidence instead of a single keyword rule.

```
User message / webpage / VPA / URL<br/> |<br/> v<br/>Browser Extension or PWA Triage<br/> - message intelligence<br/> - URL/domain signals<br/> - page/form/urgency signals<br/> - safe-domain bypass<br/> |<br/> v<br/>Central FastAPI service<br/> - MuRIL ONNX inference<br/> - explainable fraud policy<br/> - Redis cache<br/> - incident and domain persistence<br/> |<br/> v<br/>Central analyst portal + PostgreSQL/SQLite database<br/> - incident queue<br/> - domain intelligence<br/> - analyst review: safe / malicious<br/> |<br/> v<br/>Shared protection for connected devices
```

4. Objectives

1. Detect high-risk fraud language in English, Hindi, Hinglish, and selected Indian-language transliterations.

2. Analyse suspicious URLs and webpage signals without blocking based on one fraud word alone.

3. Detect risky UPI/VPA payment lures while avoiding claims that a VPA is fraudulent merely because it exists.

4. Report high-risk events to a centralized portal.

5. Maintain a cloud-backed malicious/safe domain review database.

6. Safely warn users and auto-close only corroborated high-risk pages.

7. Give analysts an explainable, auditable incident workflow.

5. Main modules

Module	Purpose
Chrome/Chromium extension	Protects browsing sessions, evaluates page/domain signals, displays warnings, captures permitted high-risk evidence, and reports incidents.
PWA fraud triage	Lets a user check a message, VPA, or URL without installing the extension. Review/high-risk results are centrally reported as `pwa-triage` incidents.
FastAPI backend	Exposes protected APIs, runs inference and policy logic, stores incidents and domain observations, and serves local demo assets.
MuRIL inference service	Runs a local ONNX MuRIL demonstration model for multilingual message scoring, supported by explainable safety signals.
Risk-policy engine	Combines model outcome, safe-domain rules, domain review status, credential fields, suspicious forms, urgency, and page context into ALLOW/REVIEW/BLOCK.
Central admin portal	Shows incidents, scores, metadata, screenshot status/hash, and domain intelligence; supports authenticated analyst review.
Database and cache	SQLite for local demonstration, PostgreSQL for cloud deployment, and Redis/Memurai for response caching.

6. How the system works

A. Browser-extension flow

1. The extension observes the active webpage after page load.

2. It first checks whether the host is on the safe-domain allowlist. Safe domains are bypassed so keywords cannot cause a false block.

3. It derives structured, privacy-limited evidence such as hostname, credential form presence, suspicious redirection, urgency language, payment lure patterns, and known domain-review status.

4. It asks the backend for a policy decision.

5. The decision is one of:

- **ALLOW:** no disruptive action;

- **REVIEW:** visible warning, but no automatic page close;

- **BLOCK:** a strong warning, high-risk incident report, and a timed circuit-breaker close unless the user explicitly overrides it.

6. The backend records the observation. The analyst may later label its domain safe or malicious. A confirmed malicious review affects future visits from connected devices.

B. PWA triage flow

1. The user enters a suspicious message, VPA, or URL in the PWA.
2. The PWA calls the backend's analysis/reputation service.
3. MuRIL inference and explainable policy indicators produce a risk result.
4. Low-risk results stay local to minimize unnecessary central reporting.
5. Review/high-risk results automatically create a central `pwa-triage` incident, even when there is no webpage URL.
6. The portal displays the score, explanation metadata, source, and time.

C. Analyst-review and shared-intelligence flow

1. An authenticated analyst opens the central portal.
2. The analyst examines the incident, URL/domain metadata, score, and allowed evidence.
3. The analyst marks a domain **safe** or **malicious** only after review.
4. A malicious decision becomes a strong policy signal for future extension and PWA checks. A safe decision prevents needless blocking.

7. Technology stack

Layer	Technology	Why it is used
Frontend	HTML5, CSS3, JavaScript, Tailwind CDN	Fast responsive PWA and professional portal UI.
Browser protection	Chrome Extension Manifest V3, service worker, content scripts, Chrome storage	Modern Chromium extension architecture with persisted server configuration.
Backend	Python, FastAPI, Uvicorn, Pydantic	High-performance typed REST API with automatic validation and documentation.
AI/NLP	Google MuRIL-compatible tokenizer/model, PyTorch training pipeline, ONNX Runtime	Multilingual Indian-language message classification with portable inference.
Explainability	Structured risk-policy rules and matched-signal output	Shows why a result is high risk; prevents opaque AI-only decisions.
Database	SQLite locally; PostgreSQL in production	Easy demo setup plus a production multi-device shared database path.
Cache	Redis; Memurai on Windows for local demo	Reduces repeated model-analysis latency without using raw message text as cache keys.
Security	JWT admin authentication, password hashing, CORS allowlist, PII redaction, input validation	Protects administrative functions and reduces sensitive-data exposure.
Deployment target	Vercel for static admin portal; hosted FastAPI service for API; managed PostgreSQL/Redis	Separates UI hosting from stateful AI/backend services.

8. Data design

Entity	Important information stored
Incident	ID, time, source, risk score, URL if available, redacted message, analysis metadata, browser/device context, screenshot hash/status.
Domain observation	Host/domain, observed risk, page signals, first/last seen time, analyst review status.
Domain review	Safe/malicious label, reason, analyst identity/time, audit entries.
Cache entry	Hashed/redacted analysis key and result expiry; raw user message is not used as the cache key.

9. Detection philosophy

GuardAIN should not make a block decision from "digital arrest", "OTP", or "UPI" alone. Those terms also occur in news, awareness campaigns, and normal conversations. A high-risk decision needs corroboration, for example:

- authority impersonation plus urgency;
- request for an OTP, UPI PIN, password, or screen sharing;
- suspicious page/form behaviour;
- risky URL construction or confirmed malicious domain status;
- payment/lure language combined with other technical evidence.

This reduces false positives and makes the project safer to demonstrate.

10. Current working features

- Professional central dashboard and PWA triage UI.
- Persistent central incident reporting from extension and PWA.
- Message-only reporting: no URL is required for a fraud-message incident.
- MuRIL ONNX model health reporting and multilingual explanation signals.
- Redis/Memurai cache status in `/health`.
- SQLite local persistence and PostgreSQL configuration support.
- Safe-domain bypass.
- Analyst-controlled safe/malicious domain reviews and audit history.
- Review versus block distinction.
- Timed auto-close only for corroborated BLOCK decisions.
- Controlled local demo pages for safe, review, credential-phishing, and UPI fraud scenarios.
- Screenshot reporting limited to high-risk visible-tab evidence when allowed.

11. Demonstration script for the panel

Opening statement (about 30 seconds)

"GuardAIN is a centralized AI-assisted fraud-defence platform. It protects a

user at the browser level, allows manual message and UPI fraud triage, and sends only meaningful high-risk reports to an analyst portal. Its key design principle is layered evidence: a fraud keyword by itself does not block a user."

Live demonstration (5–7 minutes)

1. **Show health**

- Open `http://127.0.0.1:8080/health`.

- Point out database, Redis, and MuRIL model status.

2. **Show harmless-content control**

- Open `/demo/awareness` with the extension active.

- Explain that the page uses fraud-related words but remains allowed.

- This proves the project does not work as a naive keyword blocker.

3. **Show PWA multilingual message triage**

- Open `/pwa/`.

- Paste: `CBI digital arrest notice. Share your OTP and UPI PIN now or your account will be frozen.`

- Show the high-risk explanation and central-delivery confirmation.

4. **Show the central portal**

- Open `/`, log in using the configured demonstration account.

- Refresh incidents and show `pwa-triage`, score, metadata, and timestamp.

5. **Show extension block**

- Open `/demo/extension-test`.

- Explain that page signals and phishing characteristics corroborate risk.

- Show the warning, incident attempt, and timed close. Do not choose "Proceed anyway" unless demonstrating the user override.

6. **Show domain intelligence**

- Open the domain intelligence view in the portal.

- Explain that an observed domain is not automatically convicted.

- Demonstrate that only an analyst's malicious review becomes shared block intelligence.

Closing statement

"The value of GuardAIN is not only AI classification. It combines multilingual language understanding, technical webpage evidence, privacy controls, human review, and a centralized intelligence feedback loop."

12. Test evidence

Use FRAUD_TRIAGE_TEST_CASES.md during testing. It includes MuRIL high-risk cases, Hinglish/Manglish inputs, UPI/VPA checks,

URL/extension cases, safe controls, expected results, and screenshots to capture for the report.

Recommended evidence screenshots:

1. `/health` showing database, Redis, and model status.
2. High-risk PWA message result and "central incident queued" status.
3. Portal incident with `source: pwa-triage`.
4. Extension BLOCK overlay on `/demo/extension-test`.
5. `/demo/awareness` allowed without an overlay.
6. Domain-review action and its audit entry.

13. Advantages

- **Multilingual direction:** MuRIL-based architecture supports Indian-language fraud-message handling beyond English-only rules.
- **Lower false positives:** evidence fusion and safe-domain bypass prevent blocks based solely on a word such as "OTP".
- **Centralized intelligence:** a reviewed malicious domain can help protect all devices using the same backend.
- **Human-in-the-loop:** analyst review avoids treating an AI score as final truth.
- **Explainability:** matched risk signals and policy outcome are visible.
- **Privacy-aware reporting:** routine low-risk checks are not automatically sent centrally; raw sensitive data is minimized/redacted.
- **Practical architecture:** extension, PWA, API, cache, database, and portal model a real multi-device security product.

14. Limitations and honest panel answers

Limitation	Honest explanation	Improvement path
Synthetic MuRIL dataset	The included model is a working ONNX demonstration and its local metrics are not real-world accuracy evidence.	Obtain consented, labelled, de-identified multilingual fraud/benign data; create held-out test sets and independent evaluation.
No live bank/UPI verification	The project does not and should not query bank accounts or execute payments.	Partner only through approved, consented APIs and legal agreements.
URL reputation is not proof	A suspicious URL can be legitimate and a clean-looking URL can be malicious.	Add threat-intelligence feeds, DNS/WHOIS age, certificate, redirect-chain, and screenshot similarity signals.
Extension browser scope	Manifest V3 targets Chromium-based browsers, not every device/browser.	Build Firefox/Safari adaptations and a native mobile client.
Screenshot privacy	Screenshots may contain sensitive information.	Encrypt at rest, consent gate, retention policy, access logs, and automatic redaction.
Analyst review is manual	Domain labelling requires a trained reviewer.	Add workflows, SLAs, reviewer roles, and confidence-based prioritisation.
Local demo storage	SQLite/Memurai are suitable for presentation but not a multi-region service.	Use managed PostgreSQL, managed Redis, backups, monitoring, and disaster recovery.

Limitation	Honest explanation	Improvement path
Adversarial evasion	Scammers can change spelling, domain, and page design.	Continuous retraining, URL feature engineering, feedback loops, red-team tests, and drift monitoring.

15. Risk, ethics, and privacy controls

- Never request or store a real OTP, UPI PIN, card number, password, or bank login in the demo.
- Treat model scores as advisory; use human review before shared domain blocks.
- Maintain an appeal/correction path for false-positive domain labels.
- Use HTTPS in cloud deployment and restrict CORS to trusted frontends.
- Store secrets only as deployment environment variables, never in source code.
- Apply role-based access control before public/agency deployment.
- Use data retention limits, encryption, audit logging, and consent for screenshots/evidence.
- Clearly label this as a prototype, not a government or bank product.

16. Future work roadmap

Phase 1: Production reliability

- Deploy FastAPI as a managed container/service with HTTPS.
- Replace SQLite with managed PostgreSQL and local Redis with managed Redis.
- Add automated backups, health alerts, rate limits, and structured logs.
- Add formal administrator roles and MFA.

Phase 2: Better detection

- Train MuRIL on independently labelled, consented Hindi/Hinglish/English and regional-language data.
- Measure precision, recall, F1, false-positive rate, calibration, and performance by language.
- Add URL lexical features, domain age, certificate reputation, redirect-chain analysis, phishing-kit/page similarity, and image/OCR signals.
- Add model versioning, monitoring, drift detection, and safe rollback.

Phase 3: Broader ecosystem support

- Native Android/iOS companion application.
- Firefox/Safari extension support.
- Secure, approved integration pathways for banks, CERTs, cybercrime teams, and institutional SOCs.
- Abuse-report intake, case management, analyst collaboration, and exportable

audit reports.

Phase 4: Governance

- Formal DPIA/privacy review, threat model, penetration testing, and external security audit.
- Clear false-positive appeals and data-deletion processes.
- Compliance mapping for the intended country and institution.

17. Deployment architecture for multiple laptops

For a real multi-device deployment, the portal, backend, extension, and PWA must use public HTTPS endpoints rather than `127.0.0.1`.

```
Extension/PWA on any laptop<br/> |<br/> | HTTPS API requests<br/> v<br/>Hosted FastAPI backend (public HTTPS URL)<br/> | |<br/> v v<br/>Managed PostgreSQL Managed Redis<br/> |<br/> v<br/>Vercel-hosted admin portal (HTTPS)
```

Vercel is appropriate for the static admin portal. Deploy the stateful Python AI/API component on a compatible backend host/container service, then set its public HTTPS URL in the extension and PWA configuration. Configure both frontends' CORS origins in the backend. Detailed setup files are available in `VERCEL_DEPLOYMENT.md`, `HOW_TO_IMPLEMENT_CLOUD.md`, and `BEGINNER_SETUP.md`.

18. One-line project conclusion

GuardAIN demonstrates a centralized, explainable, multilingual fraud-triage workflow that combines browser protection, AI-assisted message analysis, privacy-aware reporting, analyst review, and shared domain intelligence.